

# ARDIŞIK İŞLEMLER

## Ardışık İşlem Nedir?

Yapısal programlamada ana fonksiyondaki ilk talimatla program başlar. Python dilinde ise programın icrası kodun başındaki ilk talimattan başlar ve sırasıyla program bitene kadar sırayla devam eder. İşte programın icrasında talimat sırasını değiştirmeyen bu tür işlemlere **ardışık işlem** (**sequential operation**) adı verilir.

**Ardışık işlemler** aşağıdaki üç tür **talimattan** (**statement**) oluşur.

1. Değişkenlerin kimliklendirildiği **değişken tanımlamaları** (**identifier definition**):

```
tamsayı = 1
metin = "İlhan"
```

2. **İfadelerden** (**expression**) yani sabit, değişken ve **işleç** (**operator**) içeren sözdizimleri:

```
daireninAlanı = PI*yarıCap*yarıCap;
daireninÇevresi = 2.0*PI*yarıCap;
```

3. Klavyeden veri okuma ve konsola veri yazma gibi **giriş çıkış işlemleri** (**input output operation**):

```
print(daireninAlanı);
dosyaAdı = input()
```

## Konsoldan Girdi Okuma ve Konsola Veri Yazma

Her bilgisayar uygulamasının çalışırken kullanıcıdan girdi kabul etme olanağı olmalıdır. Bu, uygulamayı etkileşimli hale getirir. Nasıl geliştirildiğine bağlı olarak, bir uygulama kullanıcı girdisini konsola (**sys.stdin**) girilen metin, grafiksel bir düzen veya web tabanlı bir ara yüz biçiminde kabul edebilir. Bunu sağlayan **input()** fonksiyonudur. Bu fonksiyonunda okunan değerın aktarılacağı bir **dizgi** (**string**) değişken bulunmalıdır;

```
adı = input()
soyadı = input("Soyadınızı Giriniz:")
telefon = input("Telefon Numaranızı Giriniz:")
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
İlhan
Soyadınızı Giriniz:Özkan
Telefon Numaranızı Giriniz:3121234567
```

Yukarıdaki örnekte okunan telefon numarası metin bir değişkene atanmıştır. Sayıya çevirmek gerekir.

```
yarıcapstr = input('Yarıçapı Giriniz:')
yarıcap = float(yarıcapstr)

# yarıcap = float(input('Yarıçapı Giriniz:')) # şeklinde yazılabilirdi. Bu daha "Pythonic"

alan = 3.14*yarıcap*yarıcap
print('Dairenin Alanı:',alan)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
Yarıçapı Giriniz:2.5
Dairenin Alanı: 19.625
```

Konsola biçimlendirilmiş merin **print()** fonksiyonu ile yazılır. Bu fonksiyon her çağrıldığında konsola yazar ve alt satıra iner.

```
boy=180
kilo=90.5
print(boy)
# Çıktı: 180
```

```
print(boy,kilo)
# Çıktı: 180 90.5
```

Eğer birden fazla değişken var ve aralarındaki **ayraç** (**separator**) değiştirilmek isteniyorsa isimlendirilmiş **sep** parametresi değiştirilerek bu yapılabilir. Bunu isimlendirilmiş **end** parametresini değiştirerek yapabiliriz.

```
boy=180
kilo=90.5
print(boy,kilo,sep=' - ')
# Çıktı: 180 - 90.5
print(boy,'-', end='')
# Çıktı: 180- #bunu yazar ve alt satıra geçmez!
print(kilo)
# Çıktı: 180-90.5 #önce yazılan yerden devam eder!
```

## Tip Dönüşümleri

Yukarıda verilen örnekte **dizgi** (**string**) metninden kayan noktalı sayıya bir dönüşüm yapılmıştır. Benzer şekilde birçok dönüşüme ihtiyaç duyulur.

Herhangi bir dil derleyicisi/yorumlayıcısı bir veri tipindeki nesneyi otomatik olarak başka bir veri tipine dönüştürdüğünde buna otomatik veya **örtük dönüşüm** (**implicit casting**) denir. Python **güçlü veri tiplerine** (**strongly typed**) sahip bir dildir. İlgisiz veri tipleri arasında otomatik tip dönüşümüne izin vermez. Örneğin, bir **dizgi** (**string**) herhangi bir sayı tipine dönüştürülemez. Ancak, bir tam sayı (**int**), kayan noktalı bir sayıya (**float**) dönüştürülebilir.

Bir tamsayı, kayan noktalı sayıdan daha az yer kaplar. Bir tamsayı kayan noktalı sayıya örtük olarak dönüştürülebilir. Ancak bir kayan noktalı sayı, kesirli kısmı kaybolacağından, doğrudan tamsayıya dönüştürülemez.

```
tamsayı=10
kayannoktalısayı=15.5
sonuç=tamsayı+kayannoktalısayı
print(sonuç)
# Çıktı: 25.5
print(type(sonuç))
# Çıktı: <class 'float'>
bool=True # True sabitinin değeri 1, False sabitinin değeri 0'dır
sonuç=sonuç+bool
print(sonuç)
# Çıktı: 26.5
```

Otomatik veya örtük dönüşüm yalnızca **int** veri tipinden **float** veri tipine dönüşümle sınırlı olsa da **int()**, **float()** ve **str()** fonksiyonlarını kullanarak **str** veri tipinden **int** veri tipine dönüşüm gibi **açık dönüşümleri** (**explicit casting**) gerçekleştirebilirsiniz.

```
tamsayımetni='321'
tamsayı=int(tamsayımetni)
print(tamsayı)
# Çıktı: 321
kayannoktalısayımetni='7.5'
kayannoktalısayı=float(kayannoktalısayımetni)
print(kayannoktalısayı)
# Çıktı: 7.5
tamsayı=int(kayannoktalısayı)
print(tamsayı)
# Çıktı: 7
str=str(kayannoktalısayı)
print('Str:', str, 'type:', type(str))
# Çıktı: Str: 7.5 type: <class 'str'>
```

`int()` fonksiyonu ayrıca ikili, sekizli ve on altılı dizgilerden tam sayı döndürür. Bunun için fonksiyonun sırasıyla 2, 8 veya 16 olması gereken bir taban parametresine ihtiyacı vardır. Dizgi, verilen tabanda geçerli bir metni içermelidir.

```
ikilitabandan=int("1010",2)
print(ikilitabandan)
# Çıktı: 10
sekizlitabandan=int("12",8)
print(sekizlitabandan)
# Çıktı: 10
onaltiliktabandan=int("A",16)
print(onaltiliktabandan)
# Çıktı: 10
```

## İşleçler

**İşleçler** (**operator**), bir veya daha fazla **işlenen** (**operand**) üzerinde belirli işlemleri gerçekleştirmek için kullanılan özel sembollerdir. Aslında işleçler matematikten bildiğimiz işleçlerdir.

$$y = 3 + 2$$

Yukarıda verilen cebirsel ifadede toplama işleci (+) iki tarafına bulunan argümanları toplar. Bu argümanlara **işlenen** (**operand**) adı verilir. Yüksek düzey dillerde de aritmetik işleçler ve bunun yanında birçok işleç de bulunur. En çok kullanılan işleç, atama (=) işlecidir. Atama işleci, sağdaki işleneni soldakine atar. Bu nedenle kodlama yapılırken `2+2=x` veya `a+b=x` gibi sağa eşitleme yazılamaz!

İşlenenler, değişken, değer veya ifade (**expression**) olabilir. İfadelerin değişmez, değişken ve işleç içeren sözdizimleri olduğu daha önce belirtilmişti.

## Aritmetik İşleçler

Matematik ifadelerini gerçekleştirmek için kullanılır.

| İşleç | Adı                                                                                                                                       |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------|
| +     | Toplama: Sağdaki ve soldaki işlenenin toplamını verir. Eğer sadece işlenenin solunda kullanılıyorsa işleneni pozitifçe çevirir.           |
| -     | Çıkarma: Soldaki işlenenden ve sağdaki işlenenden farkını verir. Eğer sadece işlenenin solunda kullanılıyorsa işleneni negatifçe çevirir. |
| *     | Çarpma: Sağdaki ve soldaki işlenenin çarpımını verir.                                                                                     |
| /     | Bölme: Soldaki işlenenin ve sağdaki işlenene bölümünü verir.                                                                              |
| %     | <b>Kalan (modulus)</b> : Soldaki işlenenin sağdakine bölümünden kalanı verir.                                                             |
| **    | <b>Üs (exponent)</b> : Soldaki işlenenin sağdaki kadar kuvvetini verir.                                                                   |
| //    | <b>Kat bölümü (floor division)</b> : Soldaki işlenenin sağdaki işlenenin kaç katı olduğunu verir.                                         |

**Tablo 6.** Aritmetik İşleçler.

Aşağıda bu işleçler için örnek verilmiştir;

```
a=10
b=20
c = a + b
print ("a: {} b: {} a+b: {}".format(a,b,c))
# Çıktı: a: 10 b: 20 a+b: 30
c = a - b
print ("a: {} b: {} a-b: {}".format(a,b,c))
# Çıktı: a: 10 b: 20 a-b: -10
c = a * b
print ("a: {} b: {} a*b: {}".format(a,b,c))
# Çıktı: a: 10 b: 20 a*b: 200
c = b / a
print ("a: {} b: {} b/a: {}".format(a,b,c))
# Çıktı: a: 10 b: 20 b/a: 2.0
```

```

a=2
b=3
c= a**b
print ("a: {} b: {} a**b: {}".format(a,b,c))
# Çıktı: a: 2 b: 3 a**b: 8
a=30
b=6
c= a//b
print ("a: {} b: {} a//b: {}".format(a,b,c))
# Çıktı: a: 30 b: 6 a//b: 5
a = 1
sonuç = -a
print ("a: {} -a: {}".format(a,sonuç))
# Çıktı: a: 1 -a: -1

```

## Karşılaştırma İşleçleri

Karşılaştırma işleçleri (**comparision operator**) ilişkisel işleçler (**relational operator**) olarak da adlandırılır. Duruma göre seçim ve döngülerde kullanılır. Sağ ve solunda olan iki **işlenen** (**operand**) üzerinde karşılaştırma yapar.

| İşleç | Açıklama             |
|-------|----------------------|
| <     | Küçük Mü?            |
| >     | Büyük Mü?            |
| <=    | Küçük ya da Eşit Mi? |
| >=    | Büyük ya da Eşit Mi? |
| ==    | Eşit Mi?             |
| !=    | Farklı Mı?           |

**Tablo 7.** Karşılaştırma İşleçleri

Aşağıda bu işleçler için örnek verilmiştir;

```

a=5
b=6
c= a > b
print ("a: {} b: {} a>b: {}".format(a,b,c))
# Çıktı: a: 5 b: 6 a>b: False
c= a<b
print ("a: {} b: {} a<b: {}".format(a,b,c))
# Çıktı: a: 5 b: 6 a<b: True
c= a>=b
print ("a: {} b: {} a>=b: {}".format(a,b,c))
# Çıktı: a: 5 b: 6 a>=b: False
c= a<=b
print ("a: {} b: {} a<=b: {}".format(a,b,c))
# Çıktı: a: 5 b: 6 a<=b: True
c= a==b
print ("a: {} b: {} a==b: {}".format(a,b,c))
# Çıktı: a: 5 b: 6 a==b: False
c= a!=b
print ("a: {} b: {} a!=b: {}".format(a,b,c))
# Çıktı: a: 5 b: 6 a!=b: True

```

## Atama İşleçleri

Eşittir (=) sembolü **atama işleci** (**assignment operator**) olarak tanımlanır. Sağındaki işlenen değeri solundaki tek bir değişkene atar. Atama işleçleri her zaman sola atama yapar.

| İşleç | Adı                                                                   |
|-------|-----------------------------------------------------------------------|
| =     | Sağdaki işleneni sola atama yapar. <b>Soldan sağa atama yapılmaz!</b> |
| +=    | Soldaki işlenene sağdakini ekler ve sonucu sola atama yapar           |

|     |                                                                                 |
|-----|---------------------------------------------------------------------------------|
| -=  | Soldaki işlenenden sağdakini çıkarır ve sonucu sola atama yapar                 |
| *=  | Soldaki işlenenle sağdakini çarpar ve sonucu sola atama yapar                   |
| /=  | Soldaki işleneni sağdakine böler ve sonucu sola atama yapar                     |
| %=  | Soldaki işleneni sağdakine böler ve kalanı sola atama yapar                     |
| **= | Soldaki işlenenin sağdaki kadar kuvvetini alır ve sonucu sola atama yapar       |
| //= | Soldaki işlenenin sağdakinin kaç katı olduğu bulunur ve sonucu sola atama yapar |

**Tablo 8.** Atama işlemleri

Aşağıda bu işlemler için örnek verilmiştir;

```
a = 10
print(a) #10
a += 10
print(a) #20
a -= 10
print(a) #10
a *= 10
print(a) #100
a /= 10
print(a) #10.0
a %= 4
print(a) #2.0
a **= 2
print(a) #4.0
a //= 4
print(a) #1.0
```

## Mantıksal İşlemler

Mantıksal işlemler (**logical operator**) bileşik Boole ifadeleri oluşturmak için kullanılır. Bu mantıksal işlemlerin her bir işleneni kendi başına bir Boole ifadesidir. Üç mantıksal işlem vardır. Bunlar küçük harfle yazılan **and**, **or** ve **not** işlemleridir.

| İşlem      | Adı                                                                                                                                  |
|------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>and</b> | Mantıksal VE: Sağ ve soldaki işlenenler <b>True</b> ise <b>True</b> sonucunu verir. Aksi halde <b>False</b> sonucunu verir.          |
| <b>or</b>  | Mantıksal VEYA: Sağ ve soldaki işlenenlerden bir <b>True</b> ise <b>True</b> sonucunu verir. Aksi halde <b>False</b> sonucunu verir. |
| <b>not</b> | Mantıksal DEĞİL: Sağındaki işlenen <b>True</b> ise <b>False</b> sonucunu verir. Aksi halde <b>False</b> sonucunu verir.              |

**Tablo 9.** Mantıksal İşlemler

Aşağıda bu işlemler için örnek verilmiştir;

```
yaş=19
cinsiyet='E'
sonuç=yaş>18 and cinsiyet=='E'
print ("Askerlik Yapabilir: {}".format(sonuç))
# Çıktı: Askerlik Yapabilir: True
sağlıkdurumu='Engelli'
sonuç=yaş>60 or sağlıkdurumu=='Engelli'
print ("Toplu Taşımada Öncelikli: {}".format(sonuç))
# Çıktı: Toplu Taşımada Öncelikli: True
```

## Bit Düzeyi İşlemler

Bit düzeyi işlemler (**bitwise operator**) tamsayı veri tipindeki nesneler üzerinde bit seviyesinde işlemler gerçekleştirmek için kullanılır. Tamsayı veri tipindeki nesneyi bir bit dizisi olarak ele alır. Altı farklı bit düzeyi işlem vardır.

| İşleç | Adı                                                                                                        |
|-------|------------------------------------------------------------------------------------------------------------|
| &     | İki işlenen tamsayının karşılıklı olarak bitlerini VE (AND) işlemine tabi tutar.                           |
|       | İki işlenen tamsayının karşılıklı olarak bitlerini VEYA (OR) işlemine tabi tutar.                          |
| ^     | İki işlenen tamsayının karşılıklı olarak bitlerini ÖZEL VEYA (XOR) işlemine tabi tutar.                    |
| ~     | Sağdaki işlenen tamsayının her bir bitini DEĞİL (NOT) işlemine tabi tutar.                                 |
| <<    | Soldaki işlenenin bitlerini sağdaki işlenen kadar sola kaydırır. Her kaydırma 2 ile çarpma anlamına gelir. |
| >>    | Soldaki işlenenin bitlerini sağdaki işlenen kadar sağa kaydırır. Her kaydırma 2 ile bölme anlamına gelir.  |

**Tablo 10.** Bir Düzeyi İşleçler

Aşağıda bu işleçler için örnek verilmiştir. Bir tamsayıyı ikili olarak konsola yazdırmak için `bin()` fonksiyonu kullanılır;

```
a=10
b=6
sonuç=a&b
print ("a: {} b: {} a&b: {}".format(bin(a),bin(b),bin(sonuç)))
# Çıktı: a: 0b1010 b: 0b110 a&b: 0b10
sonuç=a|b
print ("a: {} b: {} a|b: {}".format(bin(a),bin(b),bin(sonuç)))
# Çıktı: a: 0b1010 b: 0b110 a&b: 0b1110
sonuç=a^b
print ("a: {} b: {} a^b: {}".format(bin(a),bin(b),bin(sonuç)))
# Çıktı: a: 0b1010 b: 0b110 a^b: 0b1100
a=2
b=1
sonuç= a<<b
print ("a: {} b: {} a<<b: {}".format(bin(a),b,bin(sonuç)))
# Çıktı: a: 0b10 b: 1 a<<b: 0b1000
sonuç= a>>b
print ("a: {} b: {} a>>b: {}".format(bin(a),b,bin(sonuç)))
# Çıktı: a: 0b10 b: 1 a>>b: 0b1
b=-1 # -1 tamsayısının bütün bitleri 1 dir
sonuç=~b
print ("b: {} ~b: {}".format(bin(b),bin(sonuç)))
# Çıktı: b: -0b1 ~b: 0b0
```

## Üyelik İşleçleri

Üyelik işleçleri (**membership operator**), bir öğenin belirli bir dizi ve koleksiyonda bulunup bulunmadığını veya üyesi olup olmadığını belirlememize yardımcı olur.

| İşleç  | Adı                                                                                |
|--------|------------------------------------------------------------------------------------|
| in     | Üye, kapsayıcı nesnede bulunursa mantıksal sonucunu <code>True</code> verir.       |
| not in | Üye, kapsayıcı nesnede bulundurmuyorsa mantıksal sonucunu <code>True</code> verir. |

**Tablo 11.** Üyelik İşleçleri

Aşağıda bu işleçler için örnek verilmiştir.

```
metin = "Bugün bayram."
bul='.'
print (bul, "in", metin, ":", bul in metin)
# Çıktı: . in Bugün bayram. : True
liste = [10,20,30,40,50]
```

```

a = 20
b = 3
print(a, "in", liste, ":", a in liste)
# Çıktı: 20 in [10, 20, 30, 40, 50] : True
print(b, "in", liste, ":", b in liste)
# Çıktı: 3 in [10, 20, 30, 40, 50] : False
print(a*b, "in", liste, ":", a*b not in liste)
# Çıktı: 60 in [10, 20, 30, 40, 50] : True

```

## Kimlik İşleçleri

**Kimlik işleçleri** (**identity operator**), nesnelerin aynı belleği paylaşıp paylaşmadıklarını ve aynı veri tipinde nesneye başvurup başvurmadıklarını belirlemek için nesneleri karşılaştırır.

| İşleç  | Adı                                                 |
|--------|-----------------------------------------------------|
| is     | Sağ ve solundaki nesne aynı nesne ise True verir.   |
| is not | Sağ ve solundaki nesne farklı nesne ise True verir. |

**Tablo 12.** Üyelik İşleçleri

Aşağıda bu işleçler için örnek verilmiştir.

```

liste1=[10,20,30,40,50]
liste2=[10,20,30,40,50]
liste3=liste1
print('liste1 ile liste2 aynı mı? ', liste1 is liste2)
# Çıktı: liste1 ile liste2 aynı mı? False
print('liste1 ile liste3 aynı mı? ', liste1 is liste3)
# Çıktı: liste1 ile liste3 aynı mı? True

```

## İşleç Öncelikleri

Cebirsel ifadelerde nasıl ki önce çarpma ve bölme sonra toplama işlemleri yapılıyor. Python programlama dilinde yazılan ifadelerde de **işleç öncelikleri** (**operator precedence**) vardır. İfadelerde bir işlemi öncelikli hale getirmek için parantez () içerisine alınır.

En içteki parantez en öncelikli olarak gerçekleştirilir. Aşağıda verilen tabloda aynı satırda olan işleçler aynı önceliğe sahiptir.

| Öncelik Sırası | İşleçler                                     |
|----------------|----------------------------------------------|
| 1              | () , [], {}                                  |
| 2              | [index]                                      |
| 3              | **                                           |
| 4              | +x, -x, ~x                                   |
| 5              | *, /, //, %                                  |
| 6              | +, -                                         |
| 7              | <<, >>                                       |
| 8              | &                                            |
| 9              | ^                                            |
| 10             |                                              |
| 11             | in, not in, is, is not, <, <=, >, >=, !=, == |
| 12             | not x                                        |
| 13             | and                                          |
| 14             | or                                           |

**Tablo 13.** İşleç Öncelikleri

Başka işlemler de bulunmaktadır bunlar ilerleyen bölümlerde yeri geldikçe anlatılacaktır<sup>11</sup>.

## Kayan Noktalı Sayı Hesapları

Kayan noktalı sayıların IEEE-754 standardında ikili tabanda tutulduğu *Değişken* başlığında anlatılmıştı. Kayan noktalı sayılar olarak değişkenlere verdiğimiz onluk tabanda değerler aslında arka planda ikilik tabanda tutulur ve bu durum yazdığımız kodlarda garip davranışların ortaya çıkmasına sebep olur. Buna örnek olarak; hemen hemen her programcının yaptığı ilk hata, aşağıdaki kodun amaçlandığı gibi çalışacağını varsaymaktır;

```
total = 0;
while total!=2.0:
    total += 0.01
    print(total)
print (total)
```

Acemi programcı bunun 0, 0.01, 0.02, 0.03, gibi artarak, 1.97, 1.98, 1.99 aralığındaki her bir sayıyı toplayacağını ve 2.0 sonucuna ulaşacağını varsayar. Bu kodda doğru yürümeyen iki şey olur: Birincisi yazıldığı haliyle program asla sonuçlanmaz. `total` asla 2'ye eşit olmaz ve döngü asla sonlanmaz. İkincisi ise döngü mantığını `total < 2.0` şartını kontrol edecek şekilde yeniden yazarsak, döngü sonlanır, ancak toplam 2.0'dan farklı bir şey olur. IEEE754 uyumlu işlemlerde, genellikle bunun yerine yaklaşık 2.0000000000000004'e ulaşır. Bunun olmasının nedeni, kayan noktalı sayıların onluk tabanda kendilerine atanan değerlerin, ikilik tabanda yaklaşık değerlerini temsil etmesidir.

```
total=0
while total<2:
    total +=0.1
print (total)
```

Örneğin program akışı aşağıdaki gibi olacaktır.

```
0.1
0.2
0.30000000000000004
0.4
0.5
0.6
0.7
0.7999999999999999
0.8999999999999999
0.9999999999999999
1.0999999999999999
1.2
1.3
1.4000000000000001
1.5000000000000002
1.6000000000000003
1.7000000000000004
1.8000000000000005
1.9000000000000006
2.0000000000000004
```

Bu duruma aşağıdaki klasik örnek verilir;

```
a=0.1
b=0.2
c=0.3
print("a: {}, b: {}, c:{}, a+b==c {}".format(a,b,c,a+b==c))
#a: 0.1, b: 0.2, c:0.3, a+b==c False
```

<sup>11</sup> [https://www.tutorialspoint.com/python/python\\_operator\\_precedence.htm](https://www.tutorialspoint.com/python/python_operator_precedence.htm)



Programcı olarak gördüğümüz şey onluk tabanda yazılmış üç sayı olsa da derleyicinin (ve altta yatan donanımın) gördüğü şey ikili sayılardır. 0.1, 0.2 ve 0.3, ona mükemmel bölmeyi gerektirdiğinden (ki bu onluk sayı sisteminde oldukça kolaydır), ancak ikili sayı sisteminde imkansızdır (bu sayılar, onluk sayı sistemindeki  $1/3$  sayısı gibi  $0.3333333333333333...$  şeklinde kesin olmayan biçimde depolanması gerektiğindendir). Bunun çözümü için aşağıdaki kod örneği verilebilir;

```
from decimal import Decimal
# Kayan nokta sorunu
print(0.1 + 0.2) # 0.30000000000000004
# Çözüm
print(Decimal('0.1') + Decimal('0.2')) # 0.3
```

## Konsola Biçimlendirilmiş Metin Yazma

Python dilinde çıktı **biçimlendirme** (**formatting**), verilerin yazdırıldığında veya kaydedildiğinde sunulma biçimini ifade eder. Doğru biçimlendirme, bilgileri daha anlaşılır ve uygulanabilir hale getirir. Python dilinde, eski tip biçimlendirmeden yeni **f-dizgisi** (**f-string**) yaklaşımına kadar birçok yöntem bulunur.

### Dizgi modül işleci

Dizgileri biçimlendirmenin en eski yollarından biri dizgi modül (%) işlecidir. Biçim belirteçlerini dizeye yerleştirerek değerleri bir dizgiye yerleştirmenize olanak tanır. Her belirteç bir % ile başlar ve biçimlendirilen değerin türünü temsil eden bir karakterle biter.

```
print("Toplam Adet: %2d, Ortalama : %5.2f" % (27, 07.444))
# Toplam Adet: 27, Ortalama : 7.44

print("Öğrenci Sayısı : %3d, Erkek : %2d, Kadın:%2d" % (98, 25, 73))
# Öğrenci Sayısı : 98, Erkek : 25, Kadın:73

print("%7o" % (64))
# 100 # sekizlik (octal) sayı sisteminde çıktı

print("%10.3E" % (3.141527))
# 3.561E+02 #bilimsel gösterim olarak biçimlendirme
```

### format() Yöntemi

Bu yöntem, dizgiye değerleri yerleştirmek için yer tutucu olarak süslü parantezler {} kullanarak dizgi interpolasyonunu daha esnek bir şekilde ele alma olanağı sağlar. Hem konumsal hem de adlandırılmış argümanları desteklediğinden, çeşitli biçimlendirme ihtiyaçları için çok yönlüdür. Dizgi metnindeki her tırnaklı parantez sırasıyla format() fonksiyonundaki parametrelere karşılık gelir.

```
print("Meyveler: {0},{1},{2}".format("Elma", "Armut","Vişne"))
#Meyveler: Elma,Armut,Vişne

print("{0} adet Elma var.".format(25))
#25 adet Elma var.

print("Pi Sayısı:{0}, Öğrenci Sayısı:{1}".format(3.1415,100))
#Pi Sayısı:3.1415, Öğrenci Sayısı:100.
```

Bu yöntem ayrıca genişlikleri, hizalamayı, sayı biçimlendirmesini ve daha fazlasını ayarlama gibi ayrıntılı biçimlendirme seçeneklerini de destekler. Tablo şeklide veriler veya raporlar oluşturmak için son derece yararlı olabilir.

```
sablon = "Toplam: {0}, Ortalama: {1} and Standart Sapma:{sapma}."
print(sablon.format(1000, 100, sapma=1.0))
# Çıktı: Toplam: 1000, Ortalama: 100 and Standart Sapma:1.0.

print("Adet: {0:4d}, Pi:{1:4.2f}".format(12, 3.141527))
```

```
# Çıktı: Adet: 12, Pi:3.14
print("Pi:{1:4.2f}, Adet: {0:4d}".format(12, 3.141527))
# Çıktı: Pi:3.14, Adet: 12

print("Adet: {adet:4d}, Pi:{pi:4.2f}".format(pi=3.141527, adet=12))
# Çıktı: Adet: 12, Pi:3.14

boy=180
kilo=90.5
print('boy:',boy,'kilo:',kilo)
# Çıktı: boy: 180 kilo: 90.5
print('boy:{}, kilo:{}'.format(boy,kilo))
# Çıktı: boy:180, kilo:90.5
print('12345678901234567890')
# Çıktı: 12345678901234567890
print('{0:>10}'.format(boy))
# Çıktı: 180
print('{0:<10}-{0:>10}'.format(boy,kilo))
# Çıktı: 180 180
```

Biçimlendirme metni aynı zamanda kendisinden sonra gelecek parametrelerin de hangi biçimde konsola yazılacağını belirler. Her bir parametrenin konsola nasıl yazılacağını **biçim belirleyicisi** (**format specifier**) belirler. Biçim belirleyicileri iki karakter olup ilk karakteri yüzde (%) karakteridir. Biçim belirleyici karakterler C dilinde olduğu gibi aşağıdakilerden biri olabilir;

| Biçim Karakterleri | Biçimlendirme metni çıktısı                                          |
|--------------------|----------------------------------------------------------------------|
| %d                 | Konsola ondalık sayı olarak yazılır                                  |
| %b                 | Konsola <b>ikili</b> ( <b>binary</b> ) olarak yazılır.               |
| %c                 | Konsola karakter olarak yazılır.                                     |
| %f                 | Konsola kayan noktalı sayı olarak yazılır.                           |
| %e                 | Konsola kayan noktalı sayı üs içerecek şekilde yazılır.              |
| %g                 | Konsola genel olarak sayı yazılır.                                   |
| %x, %X             | Konsola <b>onaltılık</b> ( <b>hexadecimal</b> ) sayı olarak yazılır. |
| %o                 | Konsola <b>sekizlik</b> ( <b>octal</b> ) sayı olarak yazılır.        |
| %s                 | Konsola metin olarak yazılır.                                        |
| %%                 | Konsola % karakteri yazılır.                                         |

**Tablo 14. Biçim Belirleyiciler**

## f-Dizgisi

Bu dizgi biçimlendirmeyi ve enterpolasyonu kolaylaştırır. f-dizgileriyle, değişkenleri ve ifadeleri süslü parantezler {} kullanarak doğrudan dizgilerin içine yerleştirebiliriz. F-dizgisi, bir dizginin başına **f** eklenir ve değişkenleri veya ifadeleri {} içine yerleştirerek kullanılır. **str.format()** gibi çalışır, ancak daha öz ve okunabilir bir şekilde.

```
boy=180
kilo=90.5
print(f"boy:{boy}, kilo: {kilo}") #boy:180, kilo: 90.5
print(f"kilo: {kilo},boy:{boy}") #kilo: 90.5,boy:180
print(f"kilo: {kilo:5.2f},boy:{boy:4d}") #kilo: 90.50,boy: 180
```

⚠ f-Dizgisinde ters bölü karakteri (\) kullanılmaz!

Ekrana tarih yazmak için aşağıdaki kod kullanılabilir. Burada **B** ay ismini, **d** rakam olarak günü ve **Y** rakam olarak yılı temsil eder.

```
import datetime
bugun = datetime.datetime.today()
print(f"{bugun:%B %d, %Y}") # October 17, 2025
```

## Hangisini Kullanmalıyız?

Konsola veya bir başka yer için metin biçimlendirirken dizgi modül işleci olan % işleci eski yöntem olup kullanılmamalıdır. `format()` yöntemi karmaşık durumlarda tercih edilmelidir. **f-dizgisi** (**f-string**) yeni nesil metin biçimlendirme olup her zaman tercih edilmelidir.

```
isim = "Ali"  
yas = 25  
print(f"{isim} {yas} yaşında")
```